二维半透膜 Gating Game 与 Phase 重定义

面向 one-target corridor 与 two-target extension 的工作备忘录

基于你现在的报告、repo 现有 phase 定义，以及你补充的 gating sketch 整理

先说结论

我建议把这件事分成两个层次来做：

1. **分类层：** 保留一个最上层的四分类，完全对齐你以前 1D ring + shortcut 的直觉：

$$\text{Excursion } E \in \{0,1\}, \qquad \text{Rollback } R \in \{0,1\}.$$

   于是得到 C00 / C10 / C01 / C11 四大类。

2. **几何层：** 不要再把 $E = 1$ 只写成抽象的 "leak"。把它细化成 **上膜 / 下膜 / 哪个 x-bin / 发生在承诺面前后**的 gate word。这样就真正变成你导师说的 gating game。

所以，**不是推翻现有 L×R，而是把 L 做成真正有空间意义的 gate system；同时把 R 做成真正的 commitment–reset 门。**

1 我对你这张补充图的理解

你给的 sketch 我理解成下面这件事：

- 主方向仍然是沿走廊从左往右推进；
- 但是在半透膜的**上方 / 下方**，你想放若干对虚拟 gate；
- 每条轨迹不再只看 "有没有 leak"，而是看它**按什么顺序穿过哪些 gate**；
- 最终目标不是一个纯形状学 phase，而是回答：**peak1、valley、peak2 分别是哪几类路径在供质量，这些路径随时间从哪里来、又往哪里去。**

这个想法我觉得是对的，而且它正好能解决你现在报告里的一个关键张力：

为什么必须做 gate，而不是只看 L0R0 / L0R1 / L1R0 / L1R1

你现在的四类已经告诉我们：在代表性 one-target 膜模型里，**peak1 主要是直接类，valley 里 leak-only 增加，peak2 却更多是 rollback-only**。这说明：**第二峰不一定等于 "漏到外面去了"**；很多慢峰其实是 "先承诺、再回撤、再重试" 的 renewal 机制。因此新的 gating game 必须把 "膜外 excursion" 和 "承诺后回撤" 作为两条正交轴来写，而不能把慢峰全都归因于 membrane leak。

2 现有结果已经告诉我们的三件事

2.1 一件事：半透膜会削弱双峰，但不会立刻摧毁它

你当前报告的 one-target corridor + membrane 结果已经很清楚：对称膜下 clear-double 会先保持一段连续带，然后进入 weak-double，再在更大渗透率下塌成单峰。这意味着机制上不是“一漏就没了”，而是**快族与慢族的时间尺度分离被逐步侵蚀**。

2.2 第二件事：第二峰不等于 leak 本身

代表案例的窗口分解已经告诉你：

- peak1：直接类占绝对主导；
- valley：直接类仍占主导，但 leak-only 明显上升；
- peak2：rollback-only 更重要，leak+rollback 只是次级修饰。

所以从物理上看，**膜的主要作用更像是改写失败尝试的时间代价，而不是单独制造一个新的吸收通道**。

2.3 第三件事：two-target 已经给了你一个“更干净”的模板

two-target 的 physics anchor 已经是一个现成模板：peak1 主要来自 near branch，peak2 主要来自 far branch。这说明 phase 的第二条如果要写好，最自然的写法不是 valley 深不深，而是：

“两峰是否由两支物理 family 在时间上分开支撑。”

这也是我下面建议 phase v2 的核心。

3 One-target membrane 的最小 gating game

3.1 先不要一上来把 gate 放得太密

最容易犯的错，是一开始就沿整条膜放很多很多小 gate，结果分类数爆炸，反而看不清。我建议第一版只做**最小可工作方案**：

1. 一个起点退出门 G_0；
2. 一个承诺门 G_q（最好来自 committor，次优可以先用近似的竖直切面）；
3. 两组膜门：一组在 G_q 左边（pre-Q），一组在 G_q 右边（post-Q）；
4. 一个 target shell 门 G_t。

这样就足够把“直接”、“先出膜再回”、“承诺后回撤”、“两者都发生”这四种大机制分开。

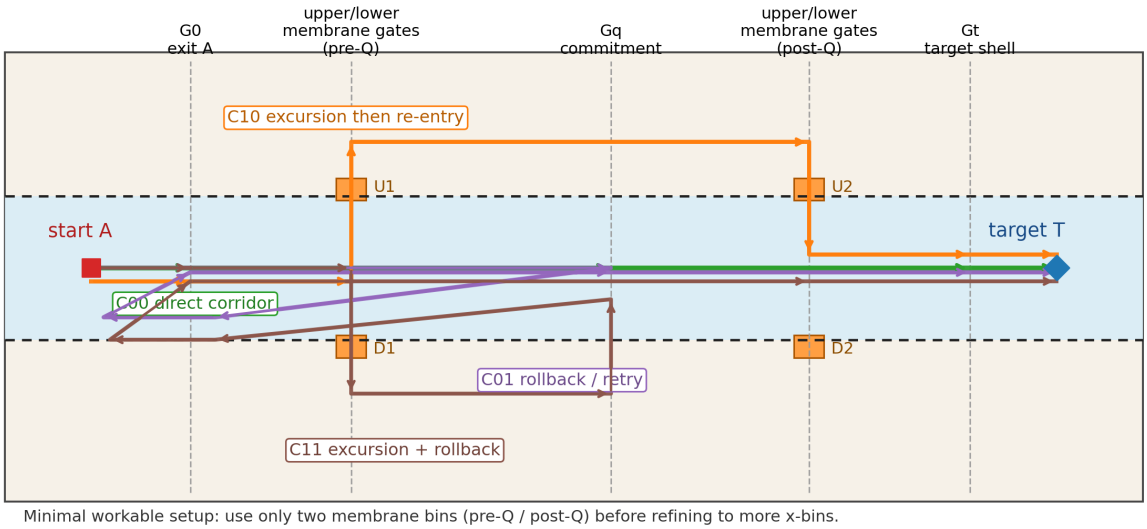

图 1: one-target 半透膜的最小 gate 布局示意。橙色小块是上/下膜门；G_q 是承诺门；G_t 是 target shell。四条彩色路径对应四大类。

3.2 门到底怎么定义

我建议你把门分成两类：

(1) 纵向推进门（progress gates）

- A：起点盆（start basin）；
- G_0：离开 A 的第一道门；
- G_q：承诺门，首选 Σ_{q^*}，其中 q^* 建议先取 0.5，之后做 0.4/0.5/0.6 鲁棒性；
- G_t：target shell，等于目标附近最后一步邻域。

(2) 膜门（membrane gates） 在两条半透膜边界上，各自放两组配对门：

$$(U_1, D_1) \quad \text{for pre-}Q, \qquad (U_2, D_2) \quad \text{for post-}Q.$$

这里 U / D 表示上侧 / 下侧。如果第一版跑顺了，再把 pre-Q 或 post-Q 各自细分成 2–3 个 x-bin。

3.3 轨迹怎么编码：不是记原始路径，而是记 reduced gate word

对每条轨迹，只记录它**第一次按顺序穿过哪些门**，并且把连续重复命中压缩掉。例如：

- 直接成功：$A \to G_0 \to G_q \to G_t$；
- 出上膜再回：$A \to G_0 \to U_1 \to U_2 \to G_q \to G_t$；
- 承诺后回撤：$A \to G_0 \to G_q \to A \to G_0 \to G_q \to G_t$；
- 出膜 + 回撤：$A \to G_0 \to D_1 \to G_q \to A \to G_0 \to G_q \to G_t$。

关键是：**你最终分析的不是格点路径，而是 gate word family。**

4 四大类：先做粗分类，再做子标签

4.1 最上层四分类

最上层我建议保留 2×2：

类	定义	物理意义
C00	$E = 0, R = 0$	全程不出膜，也没有承诺后回撤；最直接、最快的 corridor family
C10	$E = 1, R = 0$	发生过膜外 excursion，但没有完整 rollback；是 valley 的重要供给者
C01	$E = 0, R = 1$	不出膜，但承诺后回撤到 A 再重试；通常是 second peak 的主力
C11	$E = 1, R = 1$	既有 excursion，又有 rollback；通常更靠后、更拖尾

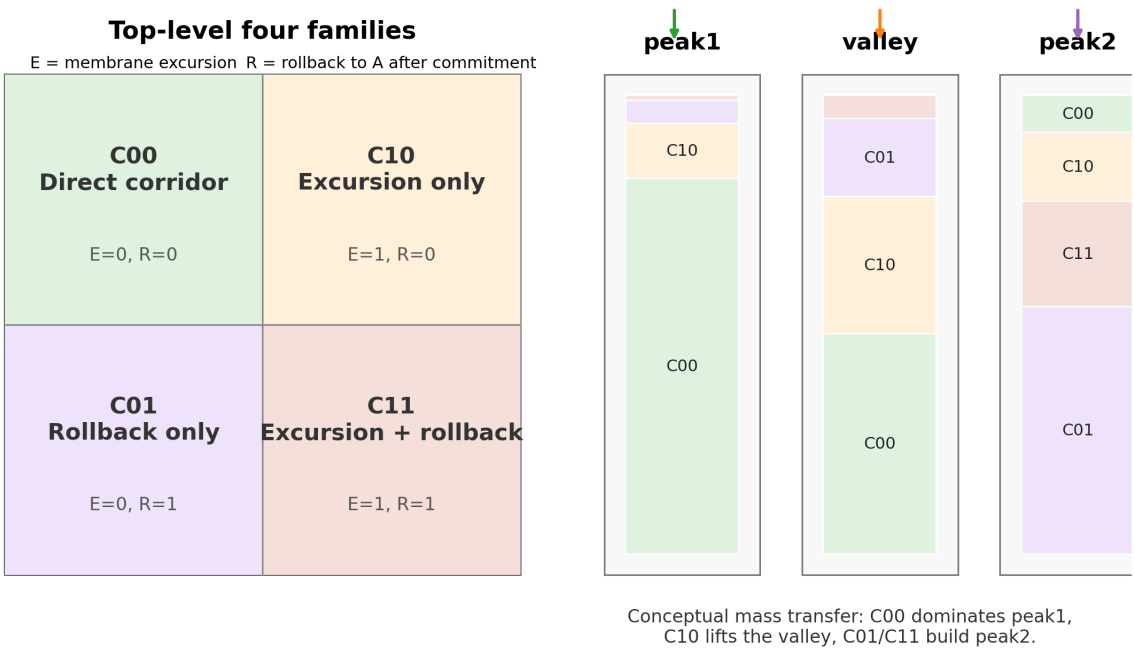

图 2: 建议保留的最上层四分类，以及一个概念化的 peak1 / valley / peak2 质量迁移图。

4.2 真正体现 gating game 的，是子标签

四大类只是**论文主图 / phase 主表面**层。真正把机制讲清楚的，是下面这些子标签：

- **side**: U / D / UD（上膜、下膜、两边都用过）；
- **bin**: pre-Q / post-Q（以后可细化成多个 x-bin）；
- **attempt count**: $0, 1, \geq 2$ 次 rollback；

- **last successful re-entry**: 最后一次成功回到走廊发生在哪个门附近。

所以你最终的分类结构应该是：

$$\text{Top-level class } (C00, C10, C01, C11) \quad + \quad \text{sub-label } (U/D,\ \text{pre/post-}Q,\ k_{\text{rollback}}).$$

这就既保住了 1D ring 那种 "四类一眼看懂" 的好处，又把 2D 膜模型真正需要的空间信息塞进去了。

5 peak1 / valley / peak2 到底对应什么空间分布

5.1 peak1：直接 corridor family

- **主导类：** C00；
- **空间上：** 质量应主要贴着走廊中心带向右推进；
- **物理上：** 几乎是首轮成功，不需要额外重试。

所以 peak1 的 occupancy 图最应该亮的是：**corridor 中轴 + 从起点到承诺门之间的推进脊线**。

5.2 valley：直接类在退场，excursion 类在入场

- **主导过渡：** C00 下降，C10 上升，C01 开始出现；
- **空间上：** 中段走廊开始变淡，上/下膜外侧出现 halo；起点盆附近可能开始重新亮起；
- **物理上：** 第一批失败尝试已经形成，但还没积累到 second peak 的主强度。

如果你把 gate-conditioned occupancy 做出来，valley 最值得看的就是：**哪一组膜门(U1/D1/U2/D2)最先亮，哪一侧先出质量**。

5.3 peak2：rollback family 真正接管，excursion+rollback 负责拖尾

- **主导类：** 通常是 C01，随后是 C11；
- **空间上：** 会同时看到起点盆重新积累，以及接近承诺门 / target shell 一带的再推进；
- **物理上：** 这是典型的 renewal picture：失败尝试若干次，最后一次才成功。

这正是为什么我不建议把慢峰直接叫成 "膜泄漏峰"。在你现在这个 one-target 代表案例里，更精确的说法应该是：

膜主要改写了失败尝试的代价；真正把 second peak 扛起来的，是 rollback / retry 家族。

6 Phase v2：只保留两条硬规则

6.1 为什么现在的第三条、第四条应该降级成 diagnostics

你 repo 里当前的 hard phase 条件，对 one-target 还要求：

$$\text{sep_peaks} \geq 1, \qquad \text{valley/max} \leq 0.35, \qquad \text{peak_balance} \geq 0.15.$$

对 two-target 还要求 near/far 质量阈值、peak ratio、peak-valley margin 等额外条件。这些条件当然能防伪双峰，但也确实会**把很多不对称但物理上真实的双峰打成 weak-double**。

我的建议是：

Phase v2 的两条硬规则

Rule 1： 总 FPT 经过现有的 `find_two_peaks` 检测后，仍然有两个可信峰。

Rule 2： 存在一对主导 gate family，它们的条件 FPT 在时间上是分开的：

$$\text{sep}_{\text{gate}} = \frac{|t_{\text{mode}}^{(\text{slow})} - t_{\text{mode}}^{(\text{fast})}|}{w_{1/2}^{(\text{fast})} + w_{1/2}^{(\text{slow})}} \geq 1.$$

这里 fast / slow 的默认选法：

- one-target：先用
$$\text{fast} = C00, \qquad \text{slow} = C10 \cup C01 \cup C11$$
作为第一版；
- 如果你发现 peak2 更纯粹地由 rollback 家族支撑，那么就改成
$$\text{fast} = C00, \qquad \text{slow} = C01 \cup C11.$$

6.2 为什么这两条就够了

因为 Rule 1 其实已经不弱了。你现有的双峰检测本身就已经包含了：

- 平滑；
- 相对高度阈值；
- 最小峰间距；
- 候选峰平衡度；
- valley drop；
- minor peak drop；
- dominance tolerance。

所以 Rule 1 已经把“尾部毛刺”、“平顶小波纹”、“肩峰假象”挡掉了一大批。在这个基础上，再加一个**机制分离**规则，就足够把真正 clear-double 和“虽然外形有两峰、但背后没有两支 family”分开。

6.3 第三条、第四条怎么处理

我的建议是：**不删掉这些量，但把它们从判决器里拿出去。**

保留为 audit / diagnostics：

- valley/max
- peak balance / peak ratio
- near/far mass（two-target）
- gate slack（新加）
- min-margin（新加）

也就是说：

$$\text{phase 只看 Rule 1 + Rule 2},$$

但图里、表里继续报告旧指标，让人知道这个 clear-double 是漂亮、一般、还是偏不对称。

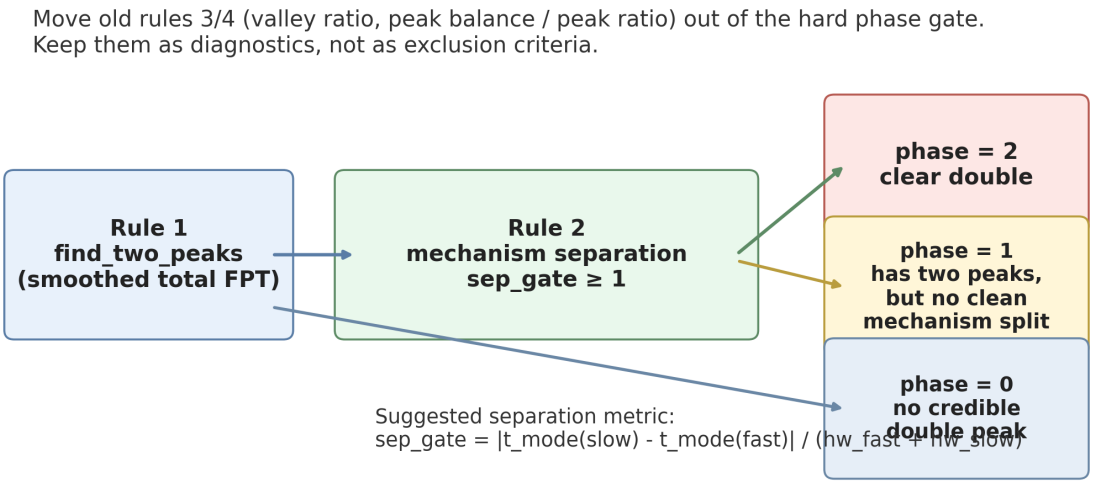

图 3: 我建议的 phase v2：旧的 valley ratio / peak balance 退出 hard gate，只保留为诊断量。

7 Two-target 怎么扩展

7.1 two-target 的好处是，本来就有一条天然 gate

对 two-target，你根本不用再围着膜放 gate；最自然的 gate 本来就是 splitting committor：

$$q_{\text{far}}(x) = \mathbb{P}_x(\tau_{\text{far}} < \tau_{\text{near}}).$$

于是

$$\Sigma_{0.5} = \{q_{\text{far}} = 0.5\}$$

就是 near / far family 的天然分界面。

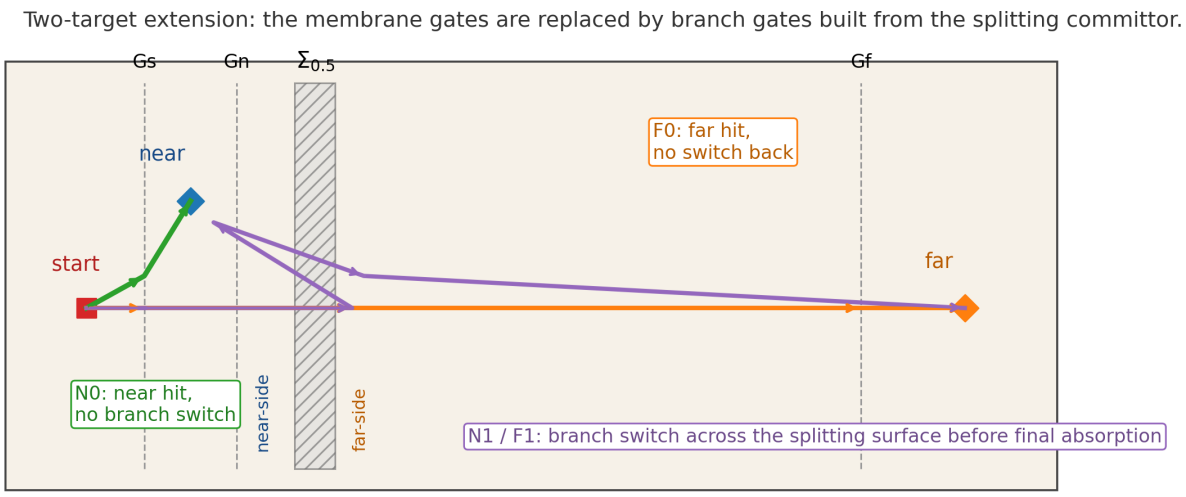

图 4: two-target 的 extension：把膜门替换成 branch gates，由 splitting committor 定义。

7.2 two-target 的四类我建议这样写

最上层写成：

$$N0,\ N1,\ F0,\ F1$$

含义分别是：

类	含义
N0	near hit，且没有跨过 splitting surface 再切到 far-side
N1	near hit，但中间发生过 branch switch / rollback
F0	far hit，且没有 switch back
F1	far hit，中间发生过 branch switch / rollback

这样 one-target 和 two-target 的逻辑就统一了：

问题	快族 / 慢族怎么分	第二条 phase 怎么写
one-target membrane	由 gate family 分	$\mathrm{sep}_{\mathrm{gate}} \geq 1$
two-target	由 near / far branch 分	$\mathrm{sep}_{\mathrm{branch}} \geq 1$

8 怎么在 repo 里落地

8.1 第一步：先做 MC 版本的 gate tagger，别一上来就 exact 扩状态

这是最快的 debug 顺序：

1. 先对现有 representative case 采样 MC 轨迹；
2. 对每条轨迹提取 reduced gate word；
3. 先把 C00/C10/C01/C11 的窗口占比做出来；

4. 确认它和你现在 L×R 的窗口图是相容的；
5. 再把上/下侧、pre/post-Q 的子标签加进去。

为什么先 MC：因为它最容易看 path atlas，最适合和你这张 sketch 对齐。

8.2 第二步：再做 exact 扩状态

等 gate 方案稳定以后，再做 exact recursion。我建议的最小扩状态是：

$$(x, y,\ e,\ r,\ b)$$

其中

- $e \in \{0, U, D, M\}$：有没有 excursion，在哪一侧；
- $r \in \{0, 1\}$：有没有 rollback；
- $b \in \{0, \text{pre-}Q, \text{post-}Q, \text{both}\}$：excursion 发生在哪个 bin。

第一版甚至可以更简化成

$$(x, y,\ E,\ R)$$

只做 top-level 四分类。等它跑顺了，再把 E 细化成 $U/D/UD$，或者再引入 pre/post-Q。

8.3 第三步：加四类图，而不是只加一张表

我建议最终至少补四类图：

1. **window-wise family mass**：peak1 / valley / peak2 各自四类占比；
2. **family-conditioned occupancy**：四类各自的空间热图；
3. **gate flux diagram**：每个窗口里各组 gate 的平均穿越次数；
4. **path atlas**：每类抽几条代表轨迹，像你以前 ring 那样直观展示。

8.4 第四步：最后再改 phase

等你已经有了 gate-conditioned FPT 后，再把

$$\text{sep}_{\text{gate}}$$

接进 phase。不要一开始先改 phase，然后再回头找 gate；顺序反过来很容易把 classification 搞乱。

9 我建议你下一步就做的最小落地清单

最小落地版（优先级从高到低）

1. 在 one-target representative 上定义 $G_0, G_q, (U_1, D_1), (U_2, D_2), G_t$；
2. 写一个 MC gate tagger，输出每条轨迹的 reduced gate word；
3. 先聚合成 C00/C10/C01/C11 四类，画 peak1 / valley / peak2 的占比图；
4. 再给 C10 / C11 分 side（U/D）；
5. 确认 peak2 的主导类到底是 C01，还是 C01+C11；
6. 定义
$$\mathrm{sep}_{\mathrm{gate}} = \frac{|t_{\mathrm{mode}}^{(\mathrm{slow})} - t_{\mathrm{mode}}^{(\mathrm{fast})}|}{w_{1/2}^{(\mathrm{fast})} + w_{1/2}^{(\mathrm{slow})}}$$
并试着在 representative case 上算出来；
7. 最后再改 one-target 的 phase 判据。

10 建议的代码骨架

下面这段不是最终实现，只是为了让你尽快把 repo 改动点想清楚：

```
@dataclass
class GateWord:
    used_excursion: bool
    used_rollback: bool
    side: str # "none", "U", "D", "UD"
    bin_tag: str # "none", "preQ", "postQ", "both"
    n_rollbacks: int

def reduce_gate_events(events) -> GateWord:
    # 1) collapse repeated hits
    # 2) detect membrane excursions by upper/lower gate hits
    # 3) detect rollback by: hit Gq, then return to A
    # 4) compress to top-level class + sub-labels
    ...

def top_level_class(word: GateWord) -> str:
    E = int(word.used_excursion)
    R = int(word.used_rollback)
    return f"C{E}{R}"

def gate_sep(f_fast, f_slow):
    t_fast = mode_time(f_fast)
    t_slow = mode_time(f_slow)
    hw_fast = half_width(f_fast, t_fast)
```

```
    hw_slow = half_width(f_slow, t_slow)
    return abs(t_slow - t_fast) / max(hw_fast + hw_slow, 1e-12)

def classify_phase_one_target_v2(summary, gate_stats):
    has_double = int(summary.t_peak1 is not None and summary.t_peak2 is not None)
    sep = gate_sep(gate_stats["fast"], gate_stats["slow"])
    return 2 if (has_double and sep >= 1.0) else (1 if has_double else 0)
```

11 一句话版总结

一句话版

你要做的不是再发明一个更复杂的 phase，而是把现有双峰写成一个“按 gate word 组织的 renewal game”：C00 管 peak1，C10 抬 valley，C01/C11 扛 peak2；phase 只保留“确实有两峰”＋“两支 gate family 在时间上真的分开”这两条硬规则。

本备忘录同时适合作为你下一轮和导师讨论的底稿。